

DMIT2015 – Lesson 5 Student Practice Activity

JSF Components + Managed Beans

Overview

In this activity, you will build a simple Jakarta Faces (JSF) student course registration form using:

- Facelets (.xhtml)
- PrimeFaces components
- Managed Beans
- Expression Language (EL)
- Validation
- Apache Tomcat deployment

This activity reinforces:

- JSF component binding
- managed bean state
- form validation
- @ViewScoped
- interactive UI updates

Learning Outcomes

By completing this activity, you should be able to:

- Create a managed bean using CDI
- Bind JSF UI components to Java properties
- Use PrimeFaces form components
- Display validation and status messages
- Deploy and test a JSF application

Estimated Time

Part 1 — Create the Managed Bean

Step 1 — Create Package

Create the following package:

```
dmit2015.view
```

Step 2 — Create CourseRegistrationBean.java

Create the following Java class:

```
CourseRegistrationBean.java
```

Step 3 — Add Required Imports

```
import jakarta.enterprise.context.ViewScoped;  
import jakarta.faces.application.FacesMessage;  
import jakarta.faces.context.FacesContext;  
import jakarta.inject.Named;
```

```
import java.io.Serializable;
```

Step 4 — Add the Class Code

Copy the following code:

```
package dmit2015.view;  
  
import jakarta.enterprise.context.ViewScoped;  
import jakarta.faces.application.FacesMessage;  
import jakarta.faces.context.FacesContext;  
import jakarta.inject.Named;  
  
import java.io.Serializable;  
  
@Named  
@ViewScoped  
public class CourseRegistrationBean implements Serializable {  
  
    private String studentName;  
    private String courseName;  
    private boolean onlineDelivery;  
  
    public void submit() {  
  
        String deliveryMode =  
            onlineDelivery ? "Online" : "In-Person";
```

```

        FacesMessage message = new FacesMessage(
            FacesMessage.SEVERITY_INFO,
            "Registration Submitted",
            studentName + " registered for "
                + courseName + " (" + deliveryMode + ")");

        FacesContext.getCurrentInstance()
            .addMessage(null, message);
    }

    public String getStudentName() {
        return studentName;
    }

    public void setStudentName(String studentName) {
        this.studentName = studentName;
    }

    public String getCourseName() {
        return courseName;
    }

    public void setCourseName(String courseName) {
        this.courseName = courseName;
    }

    public boolean isOnlineDelivery() {
        return onlineDelivery;
    }

    public void setOnlineDelivery(boolean onlineDelivery) {
        this.onlineDelivery = onlineDelivery;
    }
}

```

Part 2 — Create the JSF Page

Step 1 — Create Facelets Page

Create:

src/main/webapp/course-registration.xhtml

Step 2 — Add the XHTML Code

Copy the following code:

```
<!DOCTYPE html>

<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="jakarta.faces.html"
      xmlns:f="jakarta.faces.core"
      xmlns:p="primefaces">

<h:head>
  <title>Course Registration</title>
</h:head>

<h:body>

  <h2>DMIT2015 Course Registration</h2>

  <h:form>

    <p:messages id="messages" />

    <p:panel header="Student Registration Form">

      <p:outputLabel
        for="studentName"
        value="Student Name:" />

      <br/>

      <p:inputText
        id="studentName"
        value="#{courseRegistrationBean.studentName}"
        required="true"
        requiredMessage="Student name is required" />

      <br/><br/>

      <p:outputLabel
        for="courseName"
        value="Course:" />

      <br/>

      <p:selectOneMenu
        id="courseName"
        value="#{courseRegistrationBean.courseName}">

        <f:selectItem
          itemLabel="Select a Course"
          itemValue="" />

        <f:selectItem
          itemLabel="DMIT2015"
          itemValue="DMIT2015" />

      </p>

    </p>

  </h:form>

</h:body>

</html>
```

```
<f:selectItem
    itemLabel="CPSC1517"
    itemValue="CPSC1517" />

<f:selectItem
    itemLabel="SDEV2301"
    itemValue="SDEV2301" />

</p:selectOneMenu>

<br/><br/>

<p:selectBooleanCheckbox
    value="#{courseRegistrationBean.onlineDelivery}" />

<h:outputText
    value="Online Delivery" />

<br/><br/>

<p:commandButton
    value="Submit Registration"
    action="#{courseRegistrationBean.submit}"
    update="@form" />

</p:panel>

</h:form>

</h:body>

</html>
```

Part 3 — Deploy and Run the Application

Instructions

1. Start Apache Tomcat
2. Deploy the application
3. Open a browser
4. Navigate to:

<http://localhost:8080>

OR your project context path.

Part 4 — Test the Application

Test 1 — Missing Student Name

Leave:

Student Name

blank and click:

Submit Registration

Expected Result:

- validation message displays
- form does not submit

Test 2 — Successful Submission

Enter:

Field	Value
Student Name	Sam Wu
Course	DMIT2015
Online Delivery	Checked

Click:

Submit Registration

Expected Result:

Registration Submitted

Sam Wu registered for DMIT2015 (Online)

Test 3 — Verify View Scope

After successful submission:

- modify form values
- submit again

Verify:

- form state remains on page
- values do not reset unexpectedly

Part 5 — Required Modifications

Task 1 — Change the Page Title

Change:

DMIT2015 Course Registration

to your own custom title.

Example:

Spring 2026 Registration Form

Task 2 — Add Another Course Option

Add another course to:

```
<p:selectOneMenu>
```

Example:

```
<f:selectItem
    itemLabel="DMIT2504"
    itemValue="DMIT2504" />
```

Task 3 — Add a Footer Message

Add:

```
<h:outputText value="Powered by Jakarta Faces + PrimeFaces" />
```

below the submit button.

Bonus Challenge (Optional)

Challenge 1 — Add Email Address Field

Add:

emailAddress

property to the bean.

Add:

- input field
- getter/setter
- required validation

Challenge 2 — Add Delivery Mode Output

Display:

Delivery Mode: Online

OR

Delivery Mode: In-Person

below the submit button after registration.

Challenge 3 — Add Reset Button

Add a second button:

```
<p:commandButton value="Clear Form" />
```

that resets:

- student name
- course selection
- checkbox

Hint:

- create a `clearForm()` method in the managed bean.

Common Troubleshooting

Problem

Cannot resolve symbol primefaces

Solution

Refresh Maven dependencies.

Problem

Validation messages do not appear

Solution

Verify:

```
<p:messages />
```

exists inside:

```
<h:form>
```

Problem

Button click does nothing

Solution

Verify:

- method name matches exactly
- EL expression is correct
- bean uses:

```
@Named
```

Problem

Page state resets after submit

Solution

Verify:

```
@ViewScoped
```

and:

```
implements Serializable
```

are included.

Submission

No Brightspace submission required unless instructed by your instructor.

Be prepared to:

- demonstrate your application
- explain how binding works
- explain `@ViewScoped`
- show successful deployment and validation behavior